



Faculté d'Informatique

Multimedia Systems

Simplified MPEG-4 Video Encoder Pipeline

Mini Project — Multimedia Systems

Implementation B — Procedural / functional design

Réalisé par :

Agraine Douaa

Matricule: 222231618209

Khelifa Sirine

Matricule : 212131101651

Année Universitaire : 2025 / 2026

Table of Contents

Table of Contents	2
1. Introduction	3
2. Pipeline Description	3
2.1 Pre-processing	3
2.2 Intra-frame Coding (I-frames).....	3
2.3 Inter-frame Coding (P-frames)	3
2.4 Entropy Coding.....	3
2.5 Evaluation and Visualisation	3
3. Design Choices and Justification	4
3.1 YCbCr and 4:2:0 Chroma Subsampling	4
3.2 8×8 Block DCT using OpenCV	5
3.3 Scaled JPEG Quantisation Tables	5
3.4 Shared Motion Vectors for Chroma Components.....	5
3.5 Entropy Coding with Struct Packing and bzip2.....	5
3.6 Decoder Symmetry and Reconstruction.....	5
4. Experimental Analysis.....	5
4.1 Configuration Summary.....	6
4.2 Compression Ratio vs Quality Factor	6
5. Conclusion	6

1. Introduction

Video compression is a key part of modern multimedia systems. Every video that is streamed, recorded, or shared relies on a codec — a system that compresses raw visual data into a smaller bitstream and later reconstructs it into images.

This mini-project presents a simplified MPEG-4-like encoder pipeline implemented in Python, starting from raw image frames and producing a compressed binary file that can be decoded back into images.

This report describes a modular procedural implementation written in Python, where each stage of the encoder and decoder is implemented in a separate module/file. The generated bitstream is stored in a struct-packed format and compressed using bzip2.

2. Pipeline Description

The encoder takes a sequence of BGR image frames as input and produces a single compressed .bin bitstream. This bitstream can later be decoded to reconstruct the original video frames. The encoding process is divided into five main stages:

2.1 Pre-processing

Each BGR frame is first converted into the YCbCr color space. The chroma components (Cb and Cr) are then downsampled using a 2×2 averaging filter, producing a 4:2:0 chroma format that reduces color information while preserving visual quality.

2.2 Intra-frame Coding (I-frames)

Every G-th frame is encoded independently as an I-frame. Each image plane is centred around zero, divided into 8×8 blocks, and transformed using the DCT. The resulting coefficients are quantised using scaled JPEG quantisation tables, with separate tables for luma and chroma components.

2.3 Inter-frame Coding (P-frames)

The remaining frames are encoded as P-frames using motion prediction from the previously reconstructed frame. The luma plane is divided into 16×16 macroblocks. For each block, the encoder searches for the best match within a search range of S pixels around the current block. The prediction error (residual) is then encoded using the same DCT and quantisation process as I-frames. Chroma residuals use the same motion vectors at half resolution.

2.4 Entropy Coding

All quantised coefficients and motion vectors are serialized and compressed losslessly before being stored in the output .bin file. A decoder is implemented to reverse the complete process and reconstruct the video frames.

2.5 Evaluation and Visualisation

The system evaluates the compression performance by calculating the compression ratio, the number of I-frames and P-frames, and the PSNR of each reconstructed frame compared

to the original frames. A matplotlib figure is also generated to visualize the different stages of the encoding pipeline.

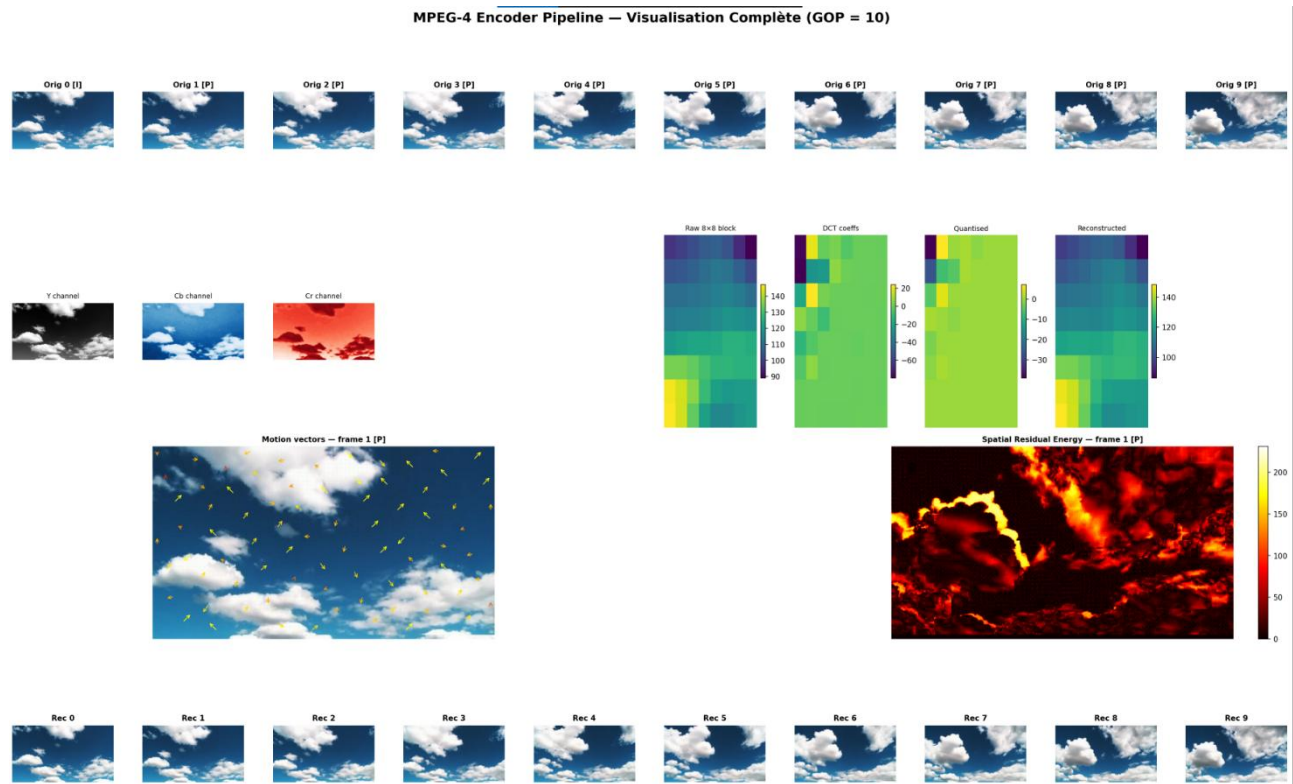


Figure 1 Single-figure visualisation of the full pipeline: original frames, YCbCr channels, 8×8 DCT & quantisation, motion vectors on a P-frame, and reconstructed frames.

3. Design Choices and Justification

3.1 YCbCr 4:2:0 Chroma Subsampling

The encoder uses the YCbCr color space with 4:2:0 chroma subsampling. This technique reduces the amount of chroma information by keeping only one quarter of the original chroma samples. Since the human eye is more sensitive to brightness than to color details, this reduction produces little visible quality loss while significantly decreasing the data size.

To perform the chroma downsampling, the encoder uses the INTER_AREA interpolation method during image resizing. This method computes each reduced pixel value from the average of the corresponding pixel area in the original image, which helps preserve visual smoothness and reduces aliasing artifacts. As a result, the chroma components are compressed efficiently while maintaining good visual quality.

3.2 8×8 Block DCT using OpenCV

The implementation uses the 2-D DCT provided by OpenCV through `cv2.dct`. This function is optimized and follows the same principles used in JPEG and MPEG compression standards. Frames are divided into 8×8 blocks because this size provides a good balance between compression efficiency and visual quality.

3.3 Scaled JPEG Quantisation Tables

Standard JPEG quantisation tables are used for both luma and chroma components. These tables apply stronger compression to high-frequency details that are less noticeable to the human eye. A quality factor is used to adjust the overall strength of quantisation, allowing the encoder to control the trade-off between compression efficiency and visual quality.

3.4 Shared Motion Vectors for Chroma Components

The chroma planes use the same motion vectors as the luma plane, scaled to half resolution because of the 4:2:0 format. This choice follows the MPEG-4 approach and reduces the computational complexity by avoiding separate motion estimation for each chroma component.

3.5 Entropy Coding with Struct Packing and bzip2

The encoded data is serialized using inline struct packing instead of Python pickle. Each array is stored with its dimensions, shape, data type, and raw bytes, making the bitstream portable and independent of Python-specific formats. The final bitstream is compressed using bzip2, which combines Burrows-Wheeler Transform, Move-to-Front coding, and Huffman coding to achieve good compression efficiency on repetitive coefficient data.

3.6 Decoder Symmetry and Reconstruction

During encoding, each P-frame is predicted using the previously reconstructed frame instead of the original one. This ensures that both the encoder and decoder use the same reference frames, preventing reconstruction drift and maintaining synchronization throughout the decoding process.

4. Experimental Analysis

Experiments were conducted on a synthetic video clip containing 10 BGR frames with a resolution of 640×360 pixels, corresponding to 35020 raw bytes. All tests and parameter sweeps were performed using the procedural encoder implementation.

4.1 Configuration Summary

Parameter	Value
Quality factor	1
GOP size	10
Macroblock size	16 × 16
DCT block size	8 × 8
Mean PSNR GOP = 10	≈ 28.54dB
Compression ratio	≈ 2.15 ×

4.2 Compression Ratio vs Quality Factor

The compression ratio decreases when the quality factor increases. At low quality settings, stronger quantisation is applied, which produces more zero coefficients after the DCT step. Since the encoded data becomes more repetitive, the compressor can reduce the file size more efficiently.

When the quality factor becomes higher, less information is removed during quantisation. As a result, more coefficients remain non-zero, increasing the amount of data stored in the bitstream and reducing the compression ratio.

The implementation uses bzip2 for entropy compression, which performs particularly well on repetitive integer coefficient data. Because of this, the obtained compression ratios are generally better than those produced by the previous implementation based on the DEFLATE method.

5. Conclusion

All main stages of the simplified MPEG-4 compression pipeline were successfully implemented and tested from start to finish. The encoder is able to convert a sequence of image frames into a single compressed .bin file, while the decoder reconstructs the video frames correctly from this bitstream.

These results confirm that the different parts of the pipeline color conversion, chroma subsampling, DCT transform, quantisation, motion estimation, entropy coding, and decoding work correctly together in a complete video compression system.